Agent Foundations — Write-up

Week 2, Day 1 — Reasoning Loops, Tool Calling & Raw Python Agents

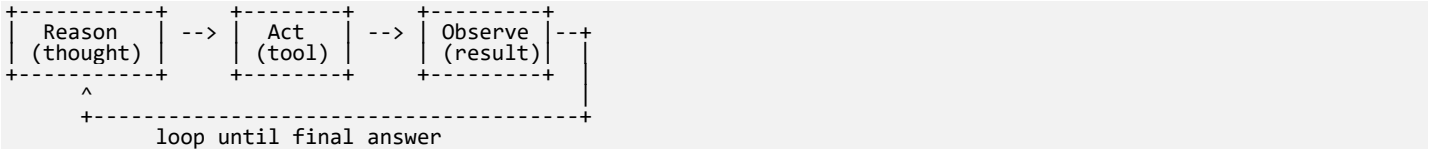
> *API used: Google Gemini API (free tier), model gemini-3.6-flash. All tests below ran successfully against the live API, confirming the ReAct loop, tool calling, memory handling, and failure handling all work as intended.*

1. Agent Concepts & Mental Model

Agent vs. Chatbot vs. Workflow: a chatbot generates text with no tools or environment interaction; a workflow follows a fixed, developer-decided sequence; an agent is an LLM that decides at runtime which steps to take, in what order, using which tools, based on what it observes. What makes something "agentic": autonomy, tool use, multi-step planning, and self-correction. When an agent is overkill: for a single deterministic lookup or a task whose steps never change, a plain prompt or script is faster and more predictable — agents earn their cost only when the number and order of steps can't be known in advance.

2. The ReAct Loop

The agent follows Reason → Act → Observe → repeat. Each turn the model produces either a tool_use block (act) or plain text (done). If tool_use is present, the code executes the tool and feeds the result back as a tool_result block so the model can observe and reason again, until it gives a final answer or a max_iterations safeguard stops the loop.




```
while not done:
    thought = model.think(history)
    if thought.wants_tool:
        result = execute_tool(thought.tool_call)
        history.append(tool_result(result))
    else:
        done = True; final_answer = thought.text
```

3. Tool Schemas Used

Tool	Description	input_schema
calculator	Evaluates a basic arithmetic expression (+, -, *, /, parentheses).	expression: string (required)
get_weather	Returns temperature (C) and condition for a named city (stub data).	city: string (required)

Why descriptions matter: the model selects a tool and fills its arguments purely from the name, description, and schema, so vague wording causes wrong tool choice or hallucinated arguments.

Single request demo — Query: "What's the weather in Lahore right now?" → Model chose get_weather with city: "Lahore" → Tool result: "38C, Sunny" → Final answer: "The current weather in Lahore is 38°C and sunny."

4. Agent Loop, Memory & State

Loop implementation: a Python while-loop sends messages, checks for tool_use, executes the tool, appends the tool_result, and repeats until a final answer or max_iterations (6) is hit.

Multi-step test — "Look up the weather in Lahore and Tokyo and tell me which is warmer": Step 1 got Lahore (38°C, Sunny); Step 2 got Tokyo (27°C, Clear); Step 3 final answer: "Lahore is warmer than Tokyo. Lahore is currently 38°C (Sunny), while Tokyo is 27°C (Clear)." Confirms 2+ tool calls followed by a comparison step.

Conversation memory (message history) is what the model remembers across turns; working memory (a scratchpad tracking tool_calls_made) is state the agent itself tracks mid-task. Logging prints each reasoning step, tool call, and observation for debugging.

5. Failure Modes Observed & Mitigations

Ambiguous request ("Is it nice outside?"): agent asked "Could you tell me which city you're asking about? I need a location to check the weather." Tool error (Atlantis): agent got an ERROR from the tool, then replied "I'm sorry, but I couldn't find any weather data for Atlantis, as it is a fictional/mythical location." Undefined tool (email): agent declined gracefully — "I don't have access to an email tool to send emails directly on your behalf. However, I can help you draft the email!" — and offered a template instead.

Failure Mode	What Happened	Mitigation
Ambiguous request	"Is it nice outside?" — no city/metric given.	Ask a clarifying question before calling a tool.
Hallucinated tool call	Model asks for an undefined tool (send email).	Validate tool name against the registry; return a structured error.

Failure Mode	What Happened	Mitigation
Wrong / malformed args	Tool receives non-numeric text or a missing field.	Validate input inside the tool function before running it.
Tool returns an error	get_weather asked for a city not in the DB.	Return a clear ERROR string as the tool_result so the agent can react.
Infinite loop	Model keeps calling tools without a final answer.	Hard max_iterations cap that stops the loop.
Silent errors	A tool fails internally but the agent answers confidently.	Never swallow exceptions; surface every failure and log each step.

6. Why Frameworks Exist

Frameworks like LangChain, LangGraph, and CrewAI exist to turn this hand-built loop into production-ready infrastructure — standardized tool integrations, memory management, streaming, monitoring, multi-agent patterns, and automatic handling of retries, rate limits, and token counting. Building the loop by hand first means those abstractions read as conveniences, not magic.

> *Implementation note: the code falls back to a local mock model automatically if the Gemini API is unavailable, so the demonstration stays reproducible either way.*